

Guía Completa: Sistema Dual LiDAR Unizona (2x VL53L4CD + STM32) desde cero

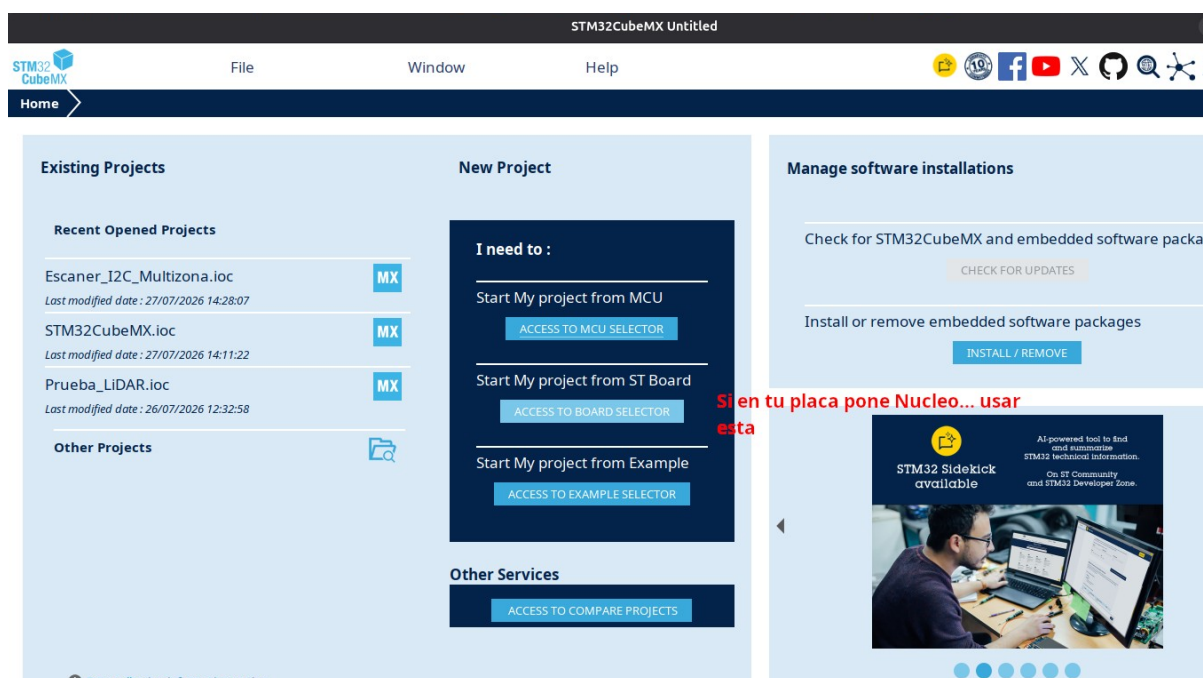
¿Solo quieres el código? Si no quieres seguir el tutorial y prefieres ir directo a la acción, puedes descargar el proyecto completo y listo para funcionar desde el repositorio de GitHub:

https://github.com/angelvaalera2/VL53L4CD_2_LiDAR

Esta guía explica paso a paso cómo se ha construido un medidor de distancia doble utilizando dos sensores ToF unizona (VL53L4CD) de forma simultánea en el mismo bus I2C, controlados por un microcontrolador STM32, resolviendo el problema de colisión de direcciones de hardware.

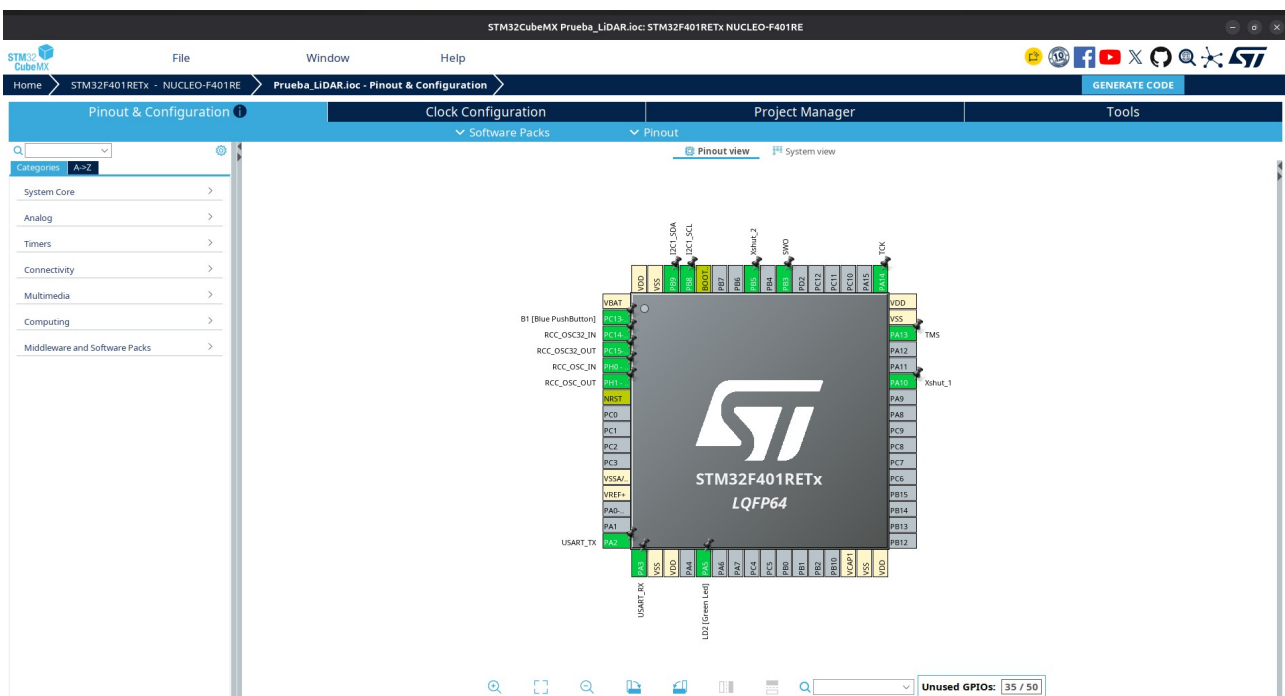
Fase 1: Configuración del Microcontrolador (STM32CubeMX)

Todo empieza configurando el "cerebro" del proyecto. Abrimos **STM32CubeMX** (o la herramienta integrada en STM32CubeIDE) y creamos un nuevo proyecto para nuestra placa (ej. STM32F401RE).





1. **Configuración del I2C:** Activamos el periférico I2C (I2C1). Asignamos los pines correspondientes para SDA (Datos) y SCL (Reloj).
2. **Pines XSHUT:** Como tenemos dos sensores idénticos, ambos traen de fábrica la misma dirección I2C (0x52). Si los conectamos sin más, chocarán. Para evitarlo, configuramos dos pines del microcontrolador como GPIO_Output (por ejemplo, PA0 y PA1). Estos pines irán conectados a los pines *XSHUT* de cada sensor para encenderlos y apagarlos a voluntad.



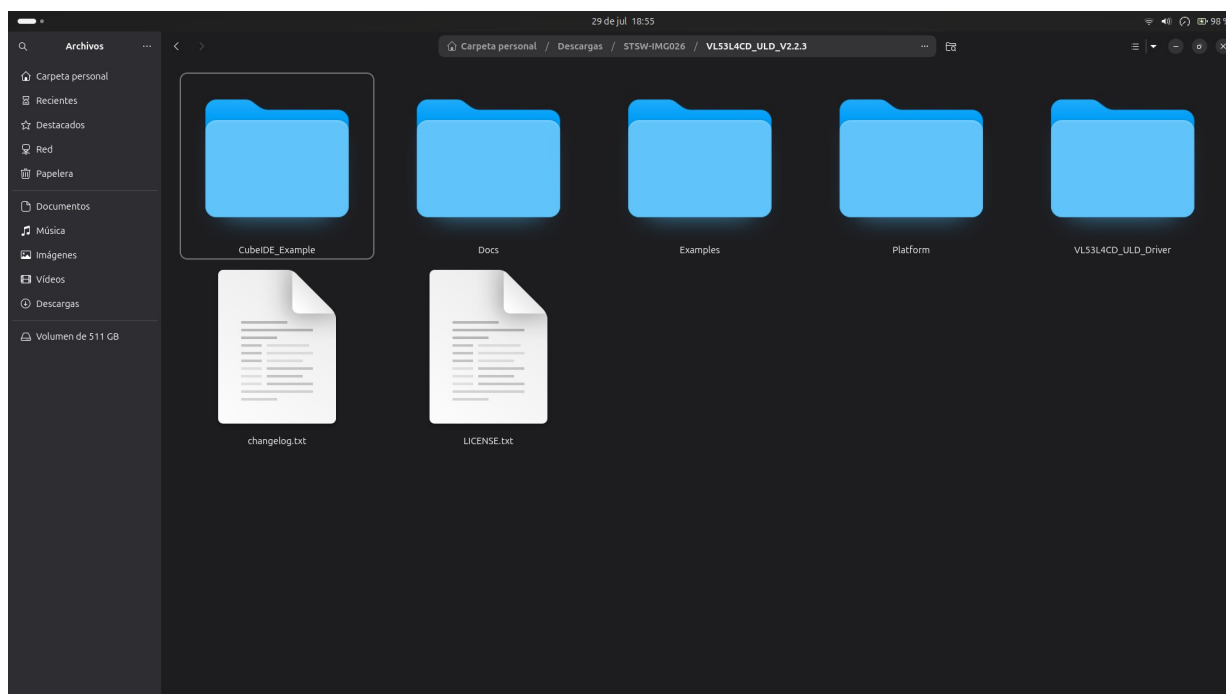
de pines (*System Core > GPIO*), selecciona los pines que asignaste al I2C (SDA y SCL), y en el menú de ajustes cambia la opción "Pull-up/Pull-down" a **Pull-up**.

4. **Configuración del puerto Serie (UART):** Nos aseguramos de tener activado el USART2 (conectado al USB de la placa Nucleo) a una velocidad de **115200 baudios**.
5. **Generar Código:** Hacemos clic en *Generate Code* para que nos cree la estructura base del proyecto.

Fase 2: Las Librerías Oficiales del Sensor

El sensor VL53L4CD es muy complejo por dentro, por lo que STMicroelectronics proporciona una API (Librería ULD).

1. Descargamos la API desde la página oficial de ST. (*Nota: Estas librerías ya están integradas si descargas mi repositorio de GitHub*).



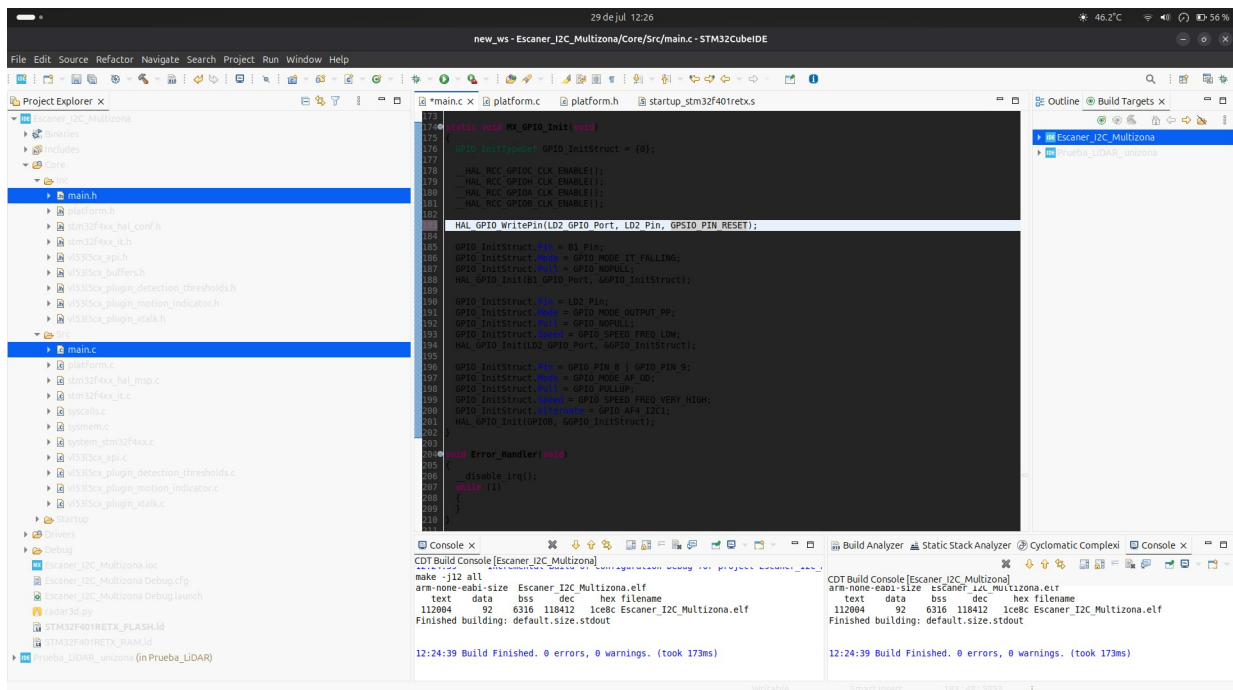
2. Copiamos los archivos .h en Core/Inc y los .c en Core/Src. (únicamente se copian los archivos de la carpeta Platform y VL53L4CD_ULD_Driver)

Fase 3: Escribiendo el Código (El cambio de dirección de el sensor)

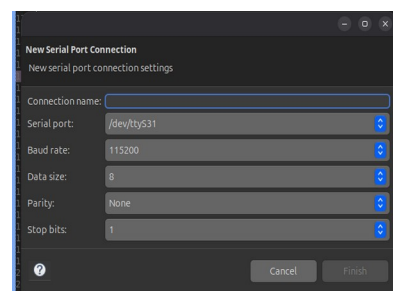
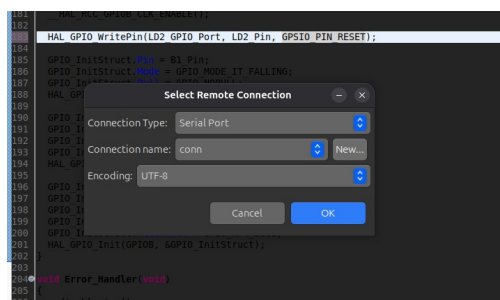
Aquí es donde se programa la lógica. Se modifican principalmente dos archivos:

- **El archivo platform.c (El Puente):** La librería oficial de ST es genérica; no sabe qué microcontrolador estás usando. En este archivo creamos un "puente". Básicamente, le dijimos a la librería: "Cuando quieras leer o escribir por I2C, utiliza estas funciones específicas de STM32 (HAL_I2C_Mem_Write y HAL_I2C_Mem_Read)".

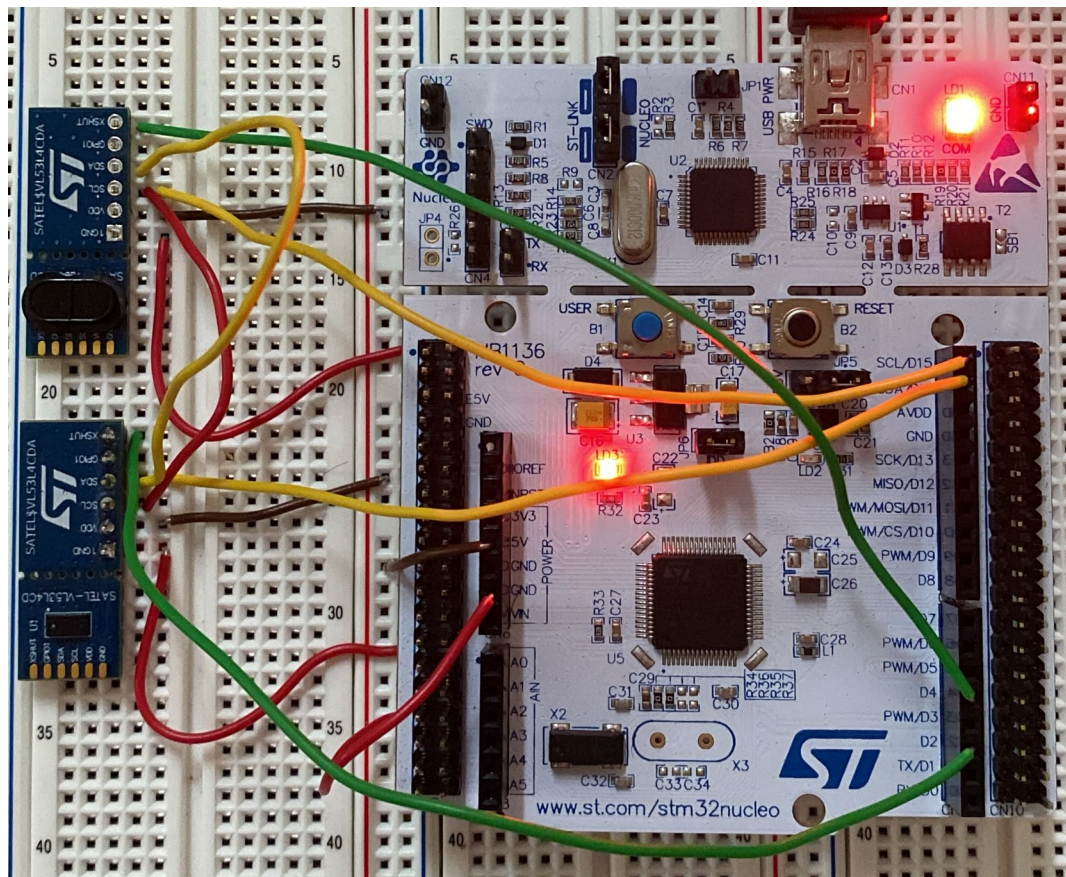
6. Dentro del bucle while(1), leemos alternativamente la distancia de ambos y las enviamos por el puerto serie usando printf.



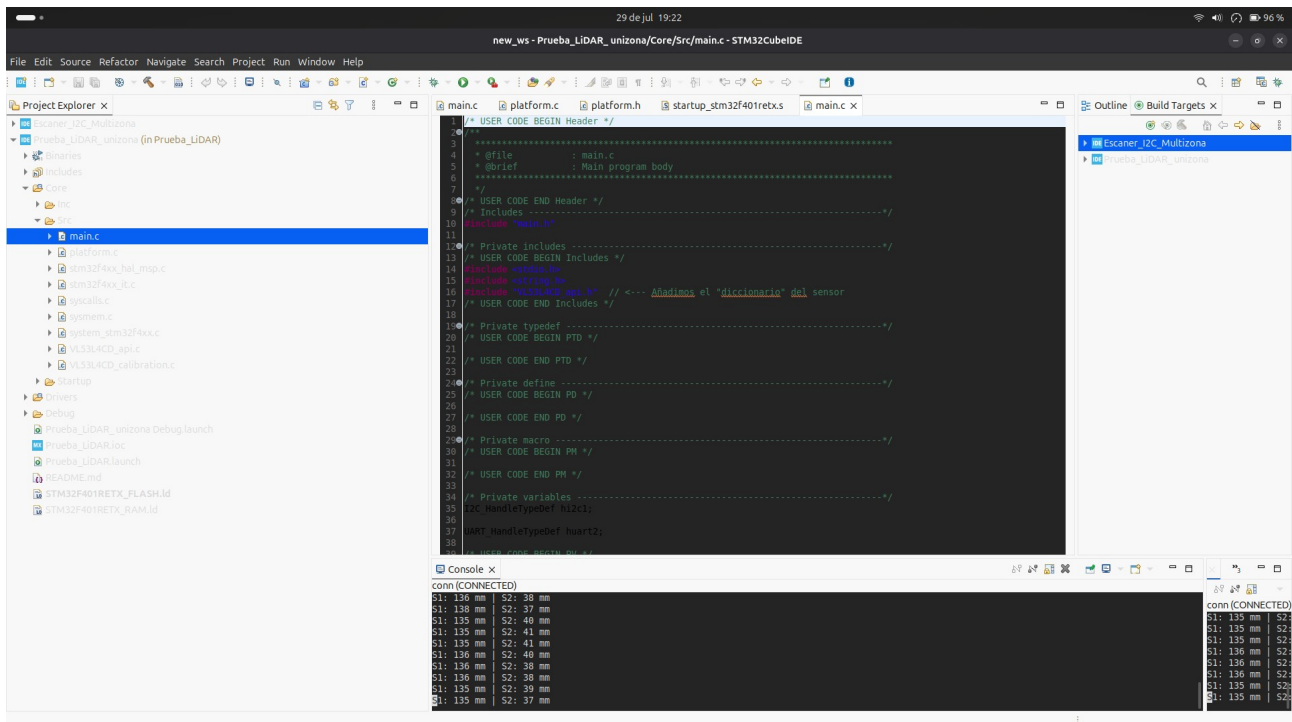
3. **Ver los datos en crudo (Opcional):** Si quieres comprobar que todo funciona antes de ir al 3D, puedes abrir una consola Serial (Clickar en la pantalla con el mas, luego Command Shell Console, al lado de connection name darle a new y conectar al serial port, normalmente es el /dev/ttyACM0) a 115200 baudios. Deberías empezar a ver líneas llenas de números separados por comas. ¡Esos son tus datos!



Fase 5: Esquema de Conexiones Hardware



- **VIN** : A los 3.3V de la placa. (ambos sensores)
- **GND**: A cualquier pin GND. (ambos sensores)
- **SDA y SCL**: A los pines I2C configurados en STM32CubeMX (los cables van en paralelo a ambos sensores).
- **XSHUT (Sensor 1)**: Al pin GPIO de salida 1 (ej. PA10, D2 en la placa NUCLEO).
- **XSHUT (Sensor 2)**: Al pin GPIO de salida 2 (ej. PB5, D4 en la placa NUCLEO).



! Fase 6: Precisión y Solución de Problemas (Troubleshooting)

A la hora de trabajar con este sistema dual, es importante que conozcas estos detalles técnicos:

- **El Offset constante de ~1 cm:** Notarás que el sensor a veces reporta una distancia que tiene un error de aproximadamente 1 centímetro respecto a la realidad. **Esto es normal aunque no se debe pasar por alto.** Lo importante es que este error suele ser constante. Si necesitas precisión milimétrica, simplemente debes medir una distancia conocida con una regla, ver la diferencia, y restar (o sumar) ese centímetro directamente en tu código en C antes de hacer el `printf`.

Console x	
conn (CLOSED)	
S1: 107 mm	S2: 110 mm
S1: 106 mm	S2: 114 mm
S1: 105 mm	S2: 111 mm
S1: 107 mm	S2: 113 mm
S1: 106 mm	S2: 111 mm
S1: 106 mm	S2: 113 mm
S1: 107 mm	S2: 111 mm
S1: 106 mm	S2: 114 mm
S1: 106 mm	S2: 111 mm

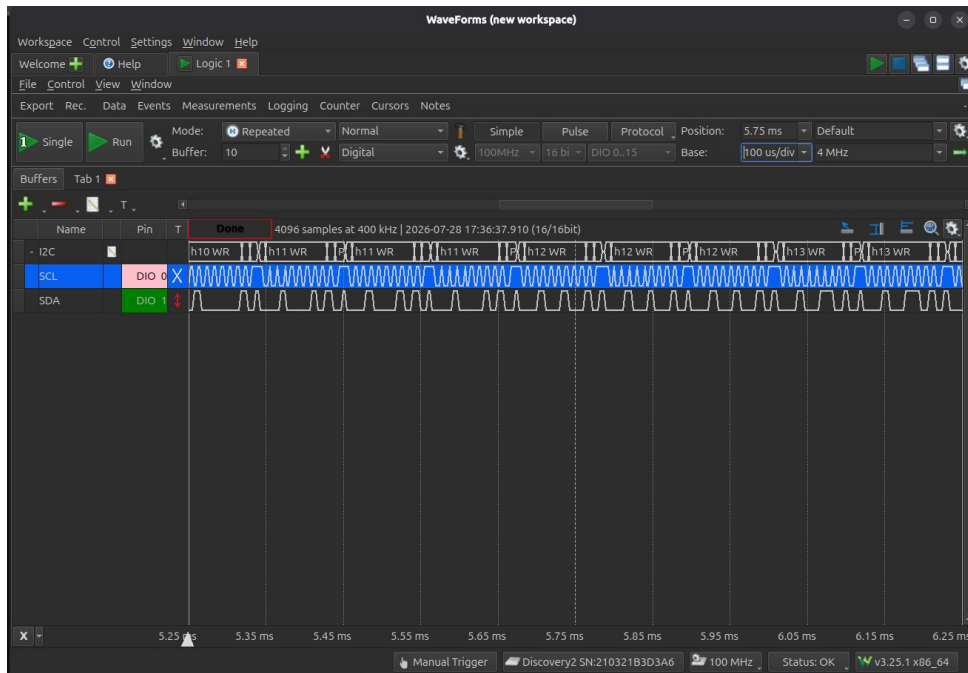
Como se aprecia en la imagen, poniendo un nivel y estando ambos sensores a la misma distancia de medida, esta difiere en algunos milímetros, por tanto en el código habría que ajustarlo para que funcione a la perfección. Además de esta diferencia entre sensores, la medida real con un metro no es la que marcan los sensores

- **Sensores que marcan lo mismo (Conflicto I2C):** Si en la consola ves que el Sensor 1 y el Sensor 2 marcan exactamente la misma distancia siempre, significa que el cambio de dirección ha fallado. Revisa que los cables del XSHUT estén bien conectados; si uno está suelto, ambos sensores se quedarán en la dirección 0x52 y el micro leerá el mismo dato dos veces.
- **Cables I2C largos:** Si usas cables muy largos para el SDA y SCL, la señal eléctrica se degrada. Intenta usar cables cortos (menos de 20 cm) o añade resistencias pull-up externas de 4.7kΩ a 3.3V si ves que el sistema se congela.

Fase 7 (Opcional): Análisis de Señales I2C (Waveforms)

Si quieres comprobar a nivel físico que el microcontrolador y el sensor se están comunicando, puedes analizar las formas de onda usando un **osciloscopio** o un **analizador lógico**.

- **Conexiones:** Conecta las sondas a los pines que hayas asignado para SDA (Datos) y SCL (Reloj). Es obligatorio conectar también la sonda de tierra al pin GND
- ¿Qué verás en pantalla?
 - **Línea SCL (Reloj):** Una onda cuadrada constante que marca el ritmo de la comunicación.
 - **Línea SDA (Datos):** Una onda irregular (saltos entre 0V y 3.3V) que representa los "unos y ceros" de los datos viajando por el cable.



¿Para qué sirve? Es la mejor forma de diagnosticar fallos. Si las líneas se quedan planas, significa que la comunicación está bloqueada o hay un cable suelto.